

Monitor the Performance of Storefront Next

Monitor the performance of Storefront Next at different stages of the app lifecycle, including in-house lab test performance and real-world usage. Create a performance benchmark to use as a future comparison basis.

Lab Data vs. Field (Real-World) Data

Collect lab data when you develop the app, and field (real-world) data after the app is deployed.

- Lab data is collected from performance tests in a controlled environment. During app development, run performance tests and collect lab data to make sure the app's performance meets performance benchmarks before it's released to customers. For performance tools to use, see [Performance Tools](#).
- Field data captures real-world user experiences. After the app is released to production, collect field data, or Real User Monitoring (RUM), from active user sessions, to understand how your app performs in production. For more information, see [Real User Monitoring](#).

Contents

[Lab Data vs. Field \(Real-World\) Data](#)

[Real User Monitoring](#)[What Are Core Web Vitals?](#)[Benchmarking and Regression Detection](#)[Performance Tools](#)[Debugging Tips and Metrics](#)



experience and measure field data, enable RUM data collection using third-party monitoring tools like Datadog, Akamai, or CrUX. With these tools, gain insights into your site's performance, including [Core Web Vitals](#).

Datadog

Datadog provides an SDK that runs in the user's browser after a storefront page loads, collecting data client-side and sending it to Datadog. Datadog RUM injects a lightweight JavaScript SDK (`@datadog/browser-rum`) into your storefront. After it's initialized with your Datadog Application ID and Client Token, the SDK runs in the user's browser and automatically captures performance metrics, user interactions, and resource timings. The SDK then sends that data directly to Datadog's intake servers.

For more information about setting up the Datadog SDK, see [RUM Browser Monitoring](#) [↗](#) in Datadog Docs.

Akamai

Akamai mPulse injects a JavaScript library, called Boomerang, into your storefront pages to collect RUM data. If your storefront is already behind Akamai's content delivery network (CDN), this injection can happen automatically at the edge, meaning no code changes are required on the storefront itself. After it loads in the user's browser, Boomerang captures Core Web Vitals, page load timings, and user interactions, then sends the data back to Akamai's mPulse platform for analysis. If your storefront doesn't use Akamai's CDN, add the Boomerang script manually via a script tag or npm package.

For more information about setting up Akamai's mPulse, see



CrUX is a public dataset from Google that provides real-world metrics from actual Chrome users and serves as the official source for Google's Web Vitals program. CrUX data includes Chrome web browsers only and doesn't include iOS devices. CrUX data is a 28-day rolling average, meaning performance improvements take time to reflect. CrUX evaluates site performance based on the 75th percentile, requiring 75% of pageviews to meet the "Good" mark for a given metric. It primarily focuses on hard navigations (initial page loads), not soft navigations within Single-Page Applications (SPAs). See [Overview of CrUX](#) in the Google Chrome documentation.

Google uses field data from CrUX for page experience ranking, not lab-based Lighthouse scores. The ultimate goal is to ensure at least 75% of real users pass Core Web Vitals thresholds.

Note

Users must meet certain criteria to have their experiences aggregated in the CrUX dataset. To learn more, see [User](#) in [CrUX methodology](#) in the Chrome for Developers documentation.

What Are Core Web Vitals?

Core Web Vitals is a set of metrics that measure real-world user experience for three key aspects of the page: loading performance, interactivity, and visual stability.

- **Largest Contentful Paint (LCP):** Measures how long it takes for the largest image or text block to appear. The goal is 2.5 seconds or less.



responsiveness to user input. The goal is 200 milliseconds or less. See [Interaction to Next Paint \(INP\)](#)

- **Cumulative Layout Shift (CLS):** Measures unexpected content shifts during loading. The goal is 0.1 or less. See [Cumulative Layout Shift](#) .

See Also

- [Understanding Core Web Vitals and Google search results](#)

Benchmarking and Regression Detection

During app development, perform initial performance testing to create a benchmark that you can compare future performance test results against.

To set up a performance test that simulates your customers' experience, know your customers' setup and choose settings that represent your average customer. Define performance goals, for example, [LCP](#) < 2.5 seconds, and track them throughout app development. If future test results show slower performance, your app has regressed. Investigate which change introduced the regression.

Use continuous integration practices to catch and fix regressions early before release. With continuous integration, code changes get merged frequently to a shared source code repository, triggering automated builds and tests. Add these tools to your development workflow, such as [Google Chrome Lighthouse](#) or [WebPageTest](#) .



to measure the performance of your app.

- **(Recommended)**
[WebPageTest](#) . A web performance testing tool with AI-powered analytics, dashboards, and real user monitoring (RUM).
- [Google Lighthouse](#) : Open-source, automated tool to help you improve the quality of web pages. It has audits for performance, accessibility, SEO, and more. To run Lighthouse during continuous integration (CI) and continuous delivery (CD) with the CLI, first run a production build with `pnpm build` and then run Lighthouse with `pnpm lighthouse:ci`. Running this command can help warn or block builds if a quality threshold isn't met.
- [PageSpeed Insights](#) : Reports on the user experience of a page on both mobile and desktop devices, and provides suggestions on how that page can be improved.
- Browser developer tools. For example, for Chrome dev tools, see [Coverage: Find unused JavaScript and CSS](#) and [Performance panel: Analyze your website's performance](#) .
- [Chrome User Experience Report \(CrUX\)](#) : Publicly available performance data per user in Chrome.

Debugging Tips and Metrics

To effectively audit and monitor performance, use these metrics and debugging tips.



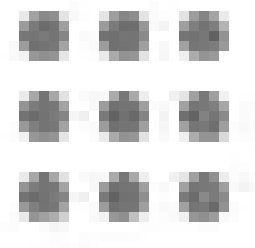
- Largest Contentful Paint (LCP)
- Interaction to Next Paint (INP)
- Cumulative Layout Shift (CLS)
- Time to First Byte (TTFB)
- Total Blocking Time (TBT)
- CDN cache hit ratio

For a description of performance metrics, see [What Are Core Web Vitals?](#) and [Time to First Byte \(TTFB\)](#). To learn more about TBT, see [Total Blocking Time \(TBT\)](#) in [web.dev](#).

B2C Commerce Metrics Dashboard for Managed Runtime (MRT)

Use the [Metrics Dashboard](#) in your B2C Commerce instance to monitor MRT metrics collected while the app is running. For example, monitor the Request Time and Cache Hits (Ratio) metric.

To access the Metrics Dashboard, in App Launcher



, select **Log Center**, and then click **Metrics**.

DID THIS
ARTICLE
SOLVE
YOUR
ISSUE?

Let us
know so

[Share your feedback](#)



Try for free



DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

C
T
E
P
B
S
S

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#)

[Contact](#) [Use of Cookies](#) [Cookie Preferences](#) [Your Privacy Choices](#)